


Welcome to Module 8: Introducing Containers and Container Services.




Section 1: Introduction.

## Module objectives

At the end of this module, you should be able to do the following:

- Describe the history, technology, and terminology behind containers
- Differentiate containers from bare-metal servers and virtual machines (VMs)
- Illustrate the components of Docker and how they interact
- Identify the characteristics of a microservices architecture
- Recognize the drivers for using container orchestration services and the AWS services that you can use for container management
- Host a dynamic website by using Docker containers
- Describe how AWS Elastic Beanstalk is used to deploy containers

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.



At the end of this module, you should be able to do the following:

- Describe the history, technology, and terminology behind containers
- Differentiate containers from bare-metal servers and virtual machines (VMs)
- Illustrate the components of Docker and how they interact
- Identify the characteristics of a microservices architecture
- Recognize the drivers for using container orchestration services and the AWS services that you can use for container management
- Host a dynamic website by using Docker containers
- Describe how AWS Elastic Beanstalk is used to deploy containers

## Module overview

### Sections

1. Introduction
2. Introducing containers
3. Introducing Docker containers
4. Using containers for microservices
5. Introducing AWS container services
6. Deploying applications with Elastic Beanstalk

### Lab

- Lab 1: Migrating a Web Application to Docker Containers
- Lab 2: Running Containers on a Managed Service

 Knowledge check

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.



This module includes the following sections:

1. Introduction
2. Introducing containers
3. Introducing Docker containers
4. Using containers for microservices
5. Introducing AWS container services
6. Deploying applications with Elastic Beanstalk

This module also includes two labs where you will work with Docker containers.

Finally, you will complete a knowledge check to test your understanding of key concepts covered in this module.

## Café business requirement

Frank and Martha recently acquired a coffee bean supplier, and they would like to include the supplier’s inventory tracking system into the café's application infrastructure. Sofía is thinking about migrating the application database to containers to complete the integration.



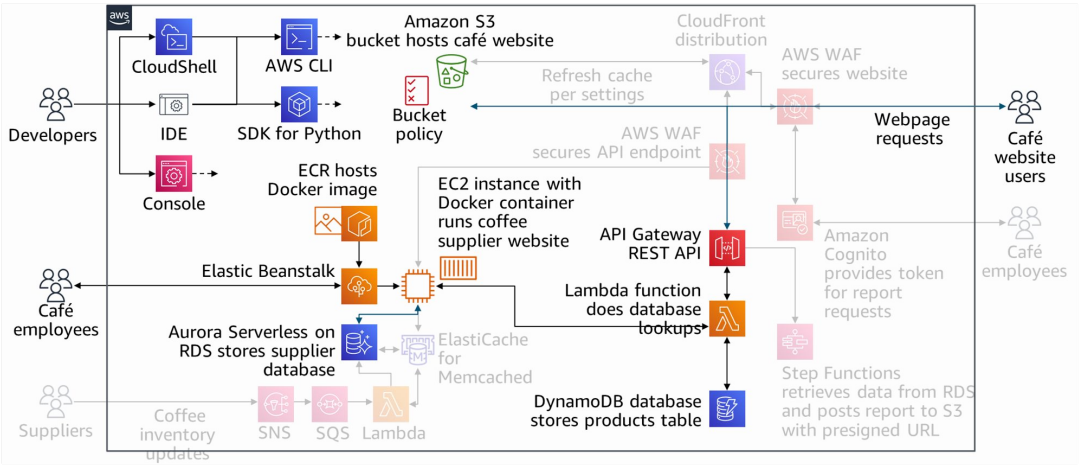


©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.



Frank and Martha recently acquired a coffee bean supplier, and they would like to include the supplier’s inventory tracking system into the café's application infrastructure. Sofía is thinking about migrating the application database to containers to complete the integration. The supplier’s inventory tracking application runs on an AWS account. Sofía has been asked to learn how the application works and then create a plan to integrate it into the café’s existing application infrastructure.

## Containers as part of developing a cloud application




©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.



The diagram on this slide gives an overview of the application that you will build through the labs in this course. The highlighted portions are relevant to this module.

As highlighted in the diagram, you will first migrate the coffee suppliers application to run on containers. Then, you will deploy the database tier with Amazon Aurora Serverless and deploy the web tier with Elastic Beanstalk.




Section 2: Introducing containers.

## Shipping containers

### Before shipping containers

- Goods were shipped in a variety of vessels with no standardized weight, shape, or size.
- Transporting goods was slow, inefficient, and costly.



### After shipping containers

- Uniformly sized shipping containers simplified loading, unloading, storing, and transferring between transport types.
- Abstraction of shipment details improved efficiency, increased productivity, and reduced costs.



aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.



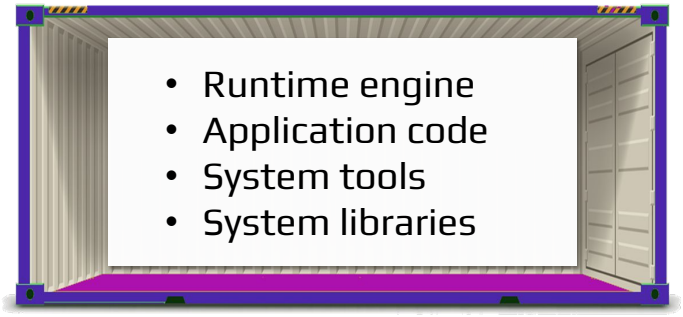
In the physical world, a *container* is a standardized unit of storage. *Container* sounds like a generic term, but its importance is clearer if you look at how containers changed the shipping industry.

Before the introduction of modern shipping containers, transporting goods from one point to another was a challenge. Goods were shipped in sacks, crates, cartons, drums, casks, barrels, and boxes of various weights, shapes, and sizes. They often were loaded by hand into whatever vessel was carrying them. The vessel wouldn't know how much cargo it could take until all of the cargo was loaded. Transporting goods this way was slow, inefficient, and costly.

Shipping containers revolutionized the shipping industry. Their uniform size made it much more efficient to load, unload, and stack them. Containers could also be easily moved between ships, trucks, and railroad cars. In this way, containers improved efficiency, increased productivity, and reduced costs.

This is a great example of using abstraction to increase agility. Abstraction, in this sense, means that the people who are involved in shipping are further removed from the details of the transaction. This abstraction applies to the people who send the contents and those who actually facilitate the shipping itself. They don't need to worry about whether items will fit in a given container or about the fit of the container onto various vessels. Abstraction is a way of hiding the working details of a subsystem. It separates concerns to facilitate interoperability and platform independence.

## A container is a standardized unit of software



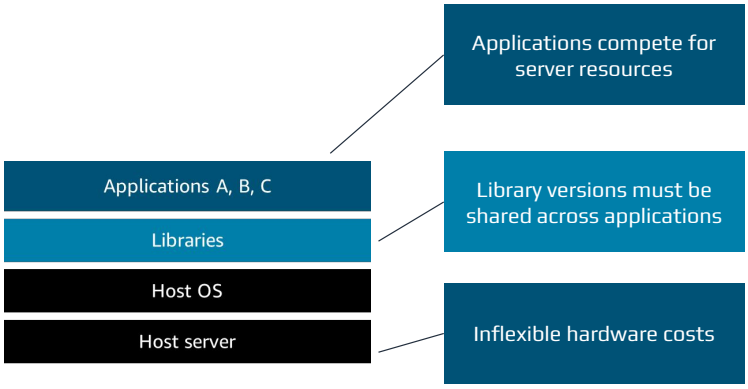

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.


This concept can move from the physical world to the virtual world of computing platforms. Virtually, a *container* is a standardized unit of software designed to run quickly and reliably on any computing environment that runs the containerization platform.

Containers provide operating system (OS) virtualization so that you can run an application and its dependencies in resource-isolated processes. A container is a lightweight, standalone software package that contains everything that a software application needs to run. For example, it can contain the application code, runtime engine, system tools, system libraries, and settings. Containers can help ensure that applications deploy quickly, reliably, and consistently regardless of the deployment environment.

A single server can host several containers that all share the underlying host system's OS kernel. These containers might be services that are part of a larger enterprise application, or they might be separate applications that run in their isolated environment.

## Evolution of deployment models: Bare-metal servers



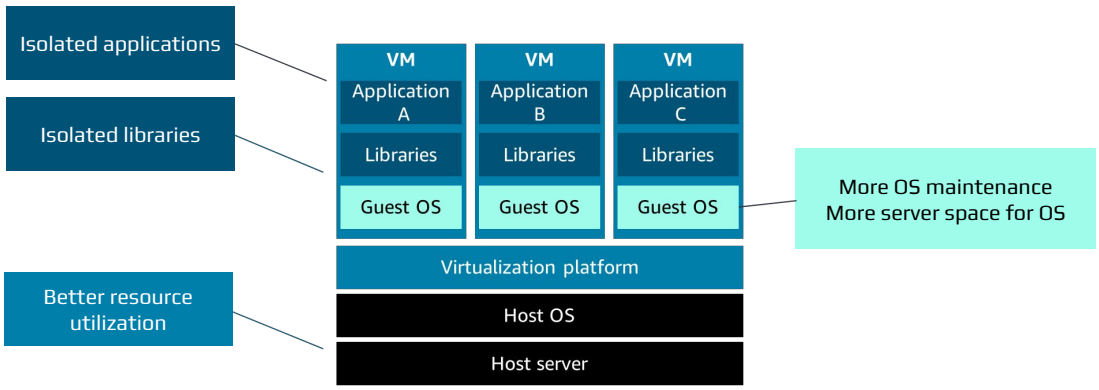

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.


Increasing levels of abstraction are often associated with technical maturity. The move toward containers is part of the evolution of deployment models.

In earlier phases, you had *bare-metal servers.* You had to build the architectural layers, such as the infrastructure and application software layers. For example, you would install an OS on top of your server hardware. Then, you would install any shared libraries on the OS, and then install the applications that use those libraries. This architecture was inefficient.

Your hardware costs are the same whether you are running at 0 percent utilization or 100 percent utilization. All of your applications must compete for the same resources, and you must keep the versions of your libraries in sync with all your applications. If one application requires an updated version of a library that is incompatible with other applications running on that host, then you run into problems.

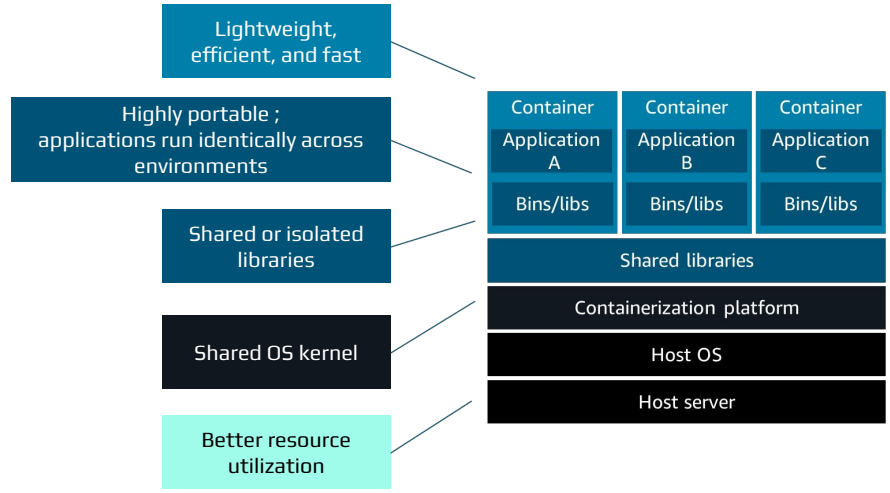
## Evolution of deployment models: VMs




aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.


These shortcomings led to the use of a virtualization platform over the host OS. Now, you have isolated applications and their libraries, which have their own full OS inside a VM. This arrangement improves utilization because you can add more VMs to run on top of the existing hardware, which greatly reduces your physical footprint. The downside to VMs is that the virtualization layer is heavy. In this example, you now have three operating systems on the physical host server, instead of one. That means more patching, more updates, and significantly more space that is taken up on the physical host. It also can cause significant redundancy: you have installed potentially the same OS three times, and potentially the same library three times.

## Evolution of deployment models: Containers




aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.


Containers improved upon the idea of virtualization. The container runtime shares the host operating system's kernel. You can use this arrangement to your advantage to create container images by using file system layers. Containers are lightweight, efficient, and fast. They can be started up and shut down faster than VMs, which means better utilization of the underlying hardware. You can share libraries when needed, but you can also have library isolation for your applications. Containers are also highly portable. Because containers isolate software from other layers, their code runs identically across different environments—from developing and staging all the way to production.

## Section 2 key takeaways



- A **container** is a standardized unit of software that contains **everything that an application** needs to run.
- Containers help to ensure that applications deploy **quickly, reliably, and consistently** regardless of the deployment environment.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.


The following are the key takeaways from this section of the module:

- A container is a standardized unit of software that contains everything that an application needs to run.
- Containers help to ensure that applications deploy quickly, reliably, and consistently regardless of the deployment environment.




Section 3: Introducing Docker containers.

## Docker container virtualization platform



Lightweight container virtualization platform



Tools to create, store, manage, and run containers



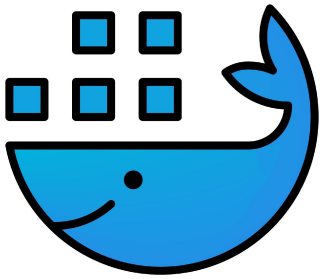
Integration with automated build, test, and deployment pipelines

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.


Containers have become popular due to the rise of Docker as a container virtualization platform. This lightweight platform provides tooling to create, store, manage, and run containers. It is easy to integrate with automated build, test, and deployment pipelines.

## Docker container benefits

- **Portable** runtime application environment
- Application and dependencies can be packaged in a **single, immutable artifact**
- Ability to run different application versions with different dependencies **simultaneously**
- **Faster** development and deployment cycles
- Better **resource utilization and efficiency**

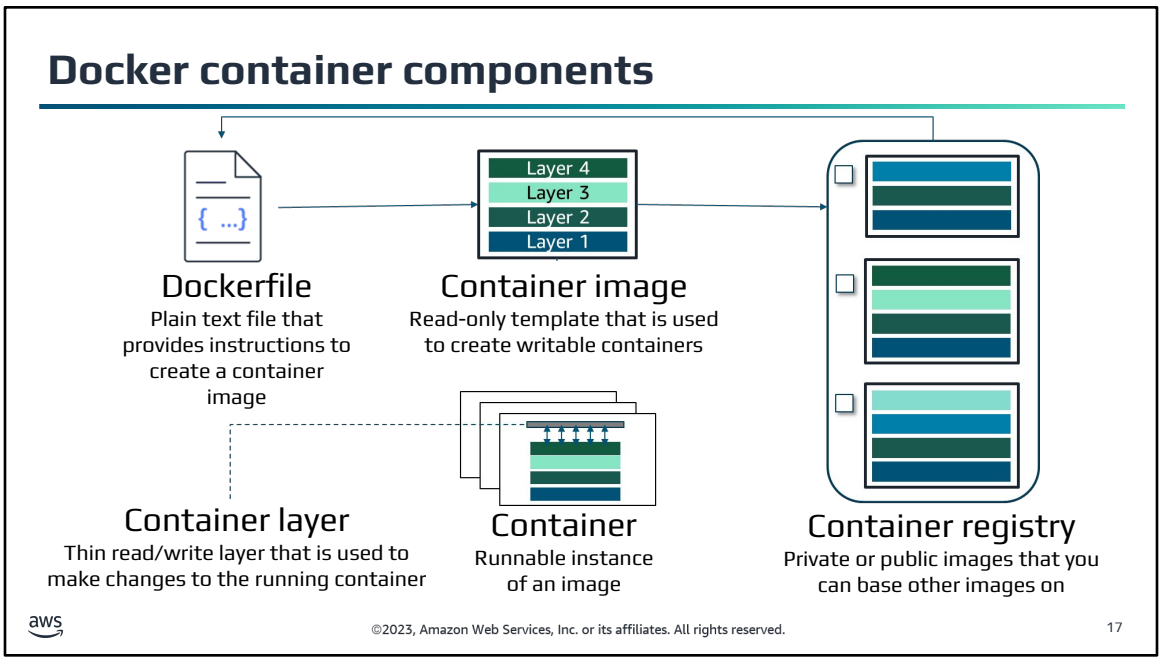


aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.


The following list summarizes some of the important benefits of Docker containers:

- Docker is a portable runtime application environment.
- You can package an application and its dependencies into a single, immutable artifact that is called an *image*.
- After you create a container image, it can go anywhere that Docker is supported.
- You can run different application versions with different dependencies simultaneously.

These benefits lead to much faster development and deployment cycles, and better resource utilization and efficiency. All of these abilities are related to agility.




Docker containers are created from read-only templates, which are called *container images*. Images are immutable and highly portable. You can port an image to any environment that supports Docker. Images are composed of layers.

Images are built from a *Dockerfile*, which is a plain text file that specifies all of the components that are included in the container. Instructions in the Dockerfile create layers in the container image.

You can create images from scratch, or you can use images that others created and published to a public or private container registry.

An image is usually based on another image, with some customization. For example, you might build an image that's based on the Ubuntu Linux image in the registry. The image could also install a web server and your application, in addition to the essential configuration details to make your application run.

A *container* is a runnable instance of an image. You can create one container or several containers based on that image.

Each container has a thin, read/write layer on top of the existing image when it is instantiated. This architecture is what makes the actual process of instantiating the containers fast. Most of the actual work is read-only because of the file system layers. The read/write layer of the container enables your applications to function properly while they are running, but it's not designed for long-term data storage. Persistent data should be stored in a volume somewhere. Consider a container as a discrete compute unit, not a storage unit.

## Dockerfile simple example

```
# Start with the Ubuntu latest image
FROM ubuntu:latest

# Output hello world message
CMD echo "Hello World!"
```

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.


To build your own image, you create a Dockerfile by using a simple syntax to define how to create the image and run it. Each instruction in a Dockerfile creates a read-only layer in the image.

In this simple example, you start with the Ubuntu latest image that is already created for you, and hosted on Docker Hub or some other site. The only thing that you do is add a command to echo the message *Hello World!* after the container is instantiated.

## Dockerfile example: Start a Java application

```
# Start with open JDK version 8 image
FROM openjdk:8

# Copy the .jar file that contains your code from your system to the
container
COPY /hello.jar /usr/src/hello.jar

# Call Java to run your code
CMD java –cp /usr/src/hello.jar
Org.example.App
```

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.


This is a slightly more involved example, in which you want to run a Java application. You start with the open Java Development Kit (JDK) version 8 image. You copy the .jar file that contains your code from your system to the container and then call Java to run your code. When this container is instantiated, it runs the Java application.

## Dockerfile example: Common tasks

```
# Start with CentOS 7 image
FROM centos:7

# Update the OS and install Apache
RUN yum –y update && yum –y install httpd

# Expose port 80—the port that the web server “listens to”
EXPOSE Port 80

# Copy shell script and give it run permissions
ADD run-httpd.sh /run-httpd.sh
RUN chmod –v +x /run-httpd.sh

# Run shell script
CMD ["/run-httpd.sh"]
```

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.


This is a more real-world example of a Dockerfile. You start with the CentOS 7 image. Next, you update the OS and install Apache. Then, you expose Port 80. Finally, you copy the shell script for your application and give it run permissions. After the container is instantiated, the command will run the shell script.

## Each line of the Dockerfile adds a layer

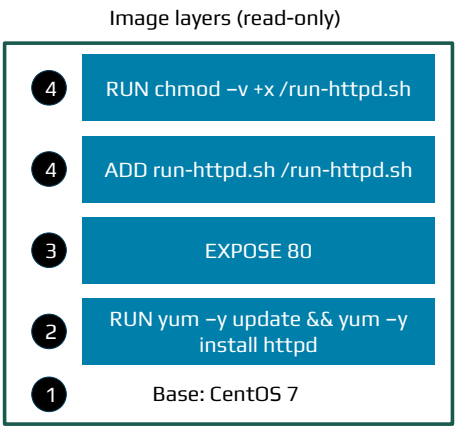
```
# 1 Start with CentOS 7 image
FROM centos:7

# 2 Update the OS and install Apache
RUN yum -y update && yum -y install httpd

# 3 Expose port 80
EXPOSE Port 80

# 4 Copy shell script and give it run
permissions
ADD run-httpd.sh /run-httpd.sh
RUN chmod -v +x /run-httpd.sh

CMD ["/run-httpd.sh"]
```




aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.


This example shows what the previous example looks like in terms of layers. Each instruction in the Dockerfile creates a layer.

- The first layer, starting from the bottom of the diagram, is the base layer
- The next layer includes the software update and the Apache installation.
- The next layer opens and exposes port 80.
- The layer after that includes the command to copy the shell script.
- Finally, the last layer makes the shell script run.

These layers are all read-only, which makes the container image an immutable object.

If you change the Dockerfile and rebuild the image, only the layers that have changed are rebuilt. This feature is part of what makes container images so lightweight, small, and fast compared to other virtualization technologies.

## Docker CLI commands

| Command | Description |
|---|---|
| `docker build` | Build an image from a Dockerfile. |
| `docker images` | List images on the Docker host. |
| `docker run` | Launch a container from an image. |
| `docker ps` | List the running containers. |
| `docker stop` | Stop a running container. |
| `docker start` | Start a container. |
| `docker push` | Push the image to a registry. |
| `docker tag` | Tag an image. |

| Command | Description |
|---|---|
| `docker logs` | View container log output. |
| `docker port` | List container port mappings. |
| `docker inspect` | Inspect container information. |
| `docker exec` | Run a command in a container. |
| `docker rm` | Remove one or more containers. |
| `docker rmi` | Remove one or more images from the host. |
| `docker update` | Dynamically update the container configuration. |
| `docker commit` | Create a new image from a container's changes. |

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.



You can run Docker command line interface (CLI) commands from a Bash terminal to manage your Docker images and containers.

For example, you can build an image from a Dockerfile by running docker build. You can then verify your image by running docker images. To launch a container from the image, run docker run. To verify that the container is running, enter *docker ps*.

After you launch a container, you can interact with it in a variety of ways. To open a Bash prompt on your running container, use docker exec. You can also stop a running container (docker stop) and start it again (docker start). To view your container logs, enter *docker logs*. To list the port mappings for your container, run docker port. Run docker tag to tag your image and use docker push to push it to a registry.

For more information about Docker CLI commands, see https://docs.docker.com/engine/reference/commandline/cli/.

## Example of docker build command

| Task | Docker command |
| --- | --- |
| Build an image from a Dockerfile in the current directory, and name the image **node_app** | `docker build --tag node_app .` |

Example output

```
Sending build context to Docker daemon  9.007MB

Step 1/7 : FROM node:11-alpine

11-alpine: Pulling from library/node
…

Successfully built a5886f101e12

Successfully tagged node_app :latest
```

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.



This example runs the `docker build` command to create an image from a Dockerfile in the current directory. The `tag` option names the new image node_app.

The actual output would show the progress of each instruction in the Dockerfile (steps 1 through 7).

## Example of docker images command

| Task | Docker command |
| --- | --- |
| List the images that your Docker client is aware of | `docker images` |

Example output

| REPOSITORY | TAG | IMAGE ID | CREATED | SIZE |
| --- | --- | --- | --- | --- |
| <none> | node_app:latest | a5886f101e12 | 18 seconds ago | 82.7MB |

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.


This example runs the docker images command to list the images that your Docker client is aware of. The actual output might include other images, but the example output highlights the image that was created with the build command in the previous slide.

The actual output would show the progress of each instruction in the Dockerfile.

## Example of docker run command

| Tasks | Docker command |
|---|---|

- Create a container named **node_app_1** from the image named **node_app**
- Run in the background and print the container ID to the terminal
- Publish container **port 8000** to the host **port 80** to make the container available to other services for HTTP

```
docker run -d --name node_app_1  -p 8000:80 node_app
```

Example output

```
5ed1ea04bcb58194100f71b2e7cd0aecab182313692ed833a6a700664994785f
```

```
docker ps
```

```
CONTAINER ID   IMAGE      COMMAND                  CREATED          STATUS          PORTS
5ed1ea04bcb5   node_app   "docker-entrypoint.s…"   9 seconds ago    Up 7 seconds    0.0.0.0:8000->80/tcp
```

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.


This example runs the docker run command to create a container from the node_app image.

The -d (or –detach) argument specifies that it should run in the background and print the container ID.

The –name argument specifies that the container should be named node_app_1.

The -p (or –publish) argument specifies to publish container port 8000 to the host port 80. By default, when you create or run a container by using docker create or docker run, it does not publish any of its ports to the outside world. The –p tag makes a port available to services outside Docker or to Docker containers that are not connected to the container's network. This technique creates a firewall rule, which maps a container port to a port on the Docker host to the outside world.

The visible output of this command is the container ID. You could follow this up with the docker ps command, as illustrated in this slide, to further verify that the container is running.

## Example of docker exec command

| Tasks | Docker command |
| --- | --- |
| • Start an **sh** terminal session on a running container<br>• List the files in the **user/src/app** directory<br>• Exit the shell session on the running container | `docker exec -it node_app_1 sh` |

Example output

```
/usr/src/app #

/usr/src/app # ls

Dockerfile          README.md           app
index.js            network.template    node_modules
package-lock.json   package.json        public              views

/usr/src/app # exit
```

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.


This example uses the docker exec command to open a terminal window on the running container. In this way, you can interact with the contents of the container.

When you run docker exec with -it <container-name> sh, Docker attempts to open an interactive connection to the shell that is running on the container. The -t specifies that you want a terminal session to be invoked. The -i option specifies interactivity, so that the terminal session that you invoke will stay open. You can continue to run commands and interact with the contents of the container until you exit the terminal.

In this simple example, after connecting to the container and opening the terminal session, you use ls to list the contents, and then exit to end the session.

## Example of docker stop and docker rm commands

| Tasks | Docker command |
| --- | --- |
| • Stop the container<br>• Remove the container | `docker stop node_app_1 && docker rm node_app_1` |

Example output

```
node_app_1
node_app_1
```

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.


This example runs the docker stop and docker rm commands to stop and then remove the container that is named node_app_1.

The visible output of this command is the name of the container for each part of the command (stop and remove). You could again follow this up with the docker container ls command to further verify that the container has been removed.

## Section 3 key takeaways



- Docker containers are created from **read-only templates**, which are called **images**.
- Images are **built from a Dockerfile** and often based on other images.
- Containers are **runnable instances** of an image with a **writable layer**.
- A container **registry** is a **repository** of images.
- To manage your Docker images and containers, you can run **Docker command line interface (CLI)** commands from a Bash terminal.

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.


The following are the key takeaways from this section of the module:

- Docker containers are created from read-only templates, which are called images.
- Images are built from a Dockerfile and often based on other images.
- Containers are runnable instances of an image with a writable layer.
- A container registry is a repository of images.
- To manage your Docker images and containers, you can run Docker CLI commands from a Bash terminal.

# Lab 8.1: Migrating a Web Application to Docker Containers



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.



You will now complete Lab 8.1: Migrating a Web Application to Docker Containers.

## Lab: Scenario

Recently, the café owners acquired one of their favorite coffee suppliers. The acquired coffee supplier runs an inventory tracking application on an AWS account.

In this lab, you again play the role of Sofía, and you will work to **migrate the application** to run on **containers**.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.



In this lab, you again play the role of Sofía, and you will work to migrate the application to run on containers.

## Lab: Tasks

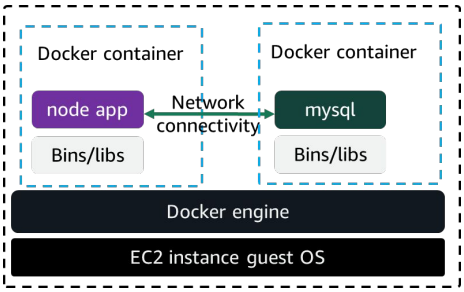
1. Preparing the development environment
2. Analyzing the existing application infrastructure
3. Migrating the application to a Docker container
4. Migrating the MySQL database to a Docker container
5. Testing the MySQL container with the node application
6. Adding the Docker images to Amazon ECR

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.


In this lab, you will complete the following tasks:

1. Preparing the development environment
2. Analyzing the existing application infrastructure
3. Migrating the application to a Docker container
4. Migrating the MySQL database to a Docker container
5. Testing the MySQL container with the node application
6. Adding the Docker images to Amazon Elastic Container Registry (Amazon ECR)

# Lab: Final product




**EC2 instance** (being used as a container host)

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.



The diagram summarizes what you will have built after you complete the lab.

Both the application and the backend data have been migrated to run on Docker containers. The containers run on an Amazon Elastic Compute Cloud (Amazon EC2) instance.




It is now time to start the lab.

# Lab debrief: Key takeaways





©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.



After you have completed the lab, your educator might choose to lead a conversation about the key takeaways from the lab.